



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Reporte Técnico de Actividades Práctico-Experimentales Nro. 4

1. Datos de Identificación del Estudiante y la Práctica

Nombre de los estudiante(s)	Tyrone Encarnación Erick Andrade Jhoan Rojas Matías Ortega
Asignatura	Matemáticas Discretas
Ciclo	1A
Unidad	Unidad 1: Lógica Matemática
Resultado de aprendizaje de la unidad	Desarrolla habilidades de razonamiento lógico para solucionar problemas ligados a la ingeniería, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001 FASE 4
Título de la Práctica	Proposiciones, Conectores Lógicos, Inferencia, Deducción y Tablas de Verdad
Nombre del Docente	Ing. Mario Enrique Cueva Hurtado
Fecha	06 de abril al 15 de mayo de 2026
Horario	Lunes, Martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	18 horas (3 horas por sesión APE)



unl

Universidad
Nacional
de Loja

2. Objetivo(s) de la Práctica

- Identificar y clasificar proposiciones lógicas simples y compuestas, distinguiendo entre proposiciones y enunciados no proposicionales.
- Comprender y aplicar los conectores lógicos fundamentales (negación, conjunción, disyunción, condicional, bicondicional) en la formulación de expresiones lógicas.
- Construir tablas de verdad para proposiciones compuestas, determinando su valor de verdad en todos los casos posibles.
- Aplicar las leyes de inferencia y deducción lógica para resolver problemas de razonamiento lógico en contextos computacionales y de ingeniería.
- Desarrollar habilidades de trabajo colaborativo, pensamiento crítico y resolución de problemas, integrando principios de equidad, inclusión y responsabilidad ética en el aprendizaje.

3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores
- Proyector multimedia
- Hojas de trabajo impresas con ejercicios de lógica proposicional y tablas de verdad
- Guía de ejercicios de inferencia y deducción lógica
- Calculadora científica
- Material bibliográfico de apoyo (textos digitalizados de referencia)
- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL para entrega de evidencias
- Entorno virtual de aprendizaje (EVA-UNL)
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes



unl

Universidad
Nacional
de Loja

4. Procedimiento / Metodología Ejecutada

FASE 4: Inferencia Lógica y Razonamiento Deductivo (Semana 4: 27 abril – 01 mayo 2026)

1. El docente presenta las reglas de inferencia lógica: Modus Ponens, Modus Tollens, Silogismo Hipotético, Silogismo Disyuntivo, Adición, Simplificación, y Conjunción, con ejemplos detallados.
2. Los estudiantes practican la identificación de reglas de inferencia en argumentos del lenguaje natural y matemático, determinando si un argumento es válido o inválido.
3. Los equipos resuelven problemas de inferencia y deducción lógica aplicando las reglas aprendidas: construir demostraciones formales paso a paso, identificar falacias lógicas comunes, y evaluar la validez de argumentos complejos.
4. Análisis guiado de implicaciones, equivalencias y razonamiento deductivo usando ejemplos computacionales: cómo las reglas de inferencia se aplican en la verificación de programas, demostración de correctitud de algoritmos e inteligencia artificial.
5. Actividad formativa (sin evaluación sumativa): Los estudiantes elaboran un resumen visual (mapa conceptual o infografía) de las reglas de inferencia estudiadas. Se habilita un foro de discusión en EVA sobre la importancia del razonamiento lógico en la ética profesional.



5. Resultados

REGLAS DE INFERENCIA

Las reglas de inferencia permiten derivar conclusiones válidas a partir de premisas. Son la base del razonamiento deductivo en lógica y en la verificación de programas.

Modus Ponens (Más usada)

$P \rightarrow q$

p

$\therefore q$

EJEMPLO EN COMPUTACIÓN	APLICACIÓN
Si el compilador no reporta errores, el programa es correcto. El compilador no reporta errores. → El programa es correcto.	Motores de inferencia, sistemas expertos, verificación

Modus Tollens (Prueba por contradicción)

$p \rightarrow q$

$\neg q$

$\therefore \neg p$

EJEMPLO EN COMPUTACIÓN	APLICACIÓN
Si hay un bug, los tests fallan. Los tests no fallan. → No hay bug.	Depuración lógica, refutación de hipótesis

Silogismo hipotético (Encadenamiento)

$p \rightarrow q$

$q \rightarrow r$

$\therefore p \rightarrow r$

EJEMPLO EN COMPUTACIÓN	APLICACIÓN
Si input es válido → se procesa. Si se procesa → se guarda. → Si input válido → se guarda.	Pipelines, flujos de datos, arquitectura por capas

Adición (Ampliación)

p

$\therefore p \vee q$

EJEMPLO EN COMPUTACIÓN	APLICACIÓN
El test pasó. → El test pasó o hay un error en otro módulo.	Sistemas de diagnóstico con alternativas



Simplificación (Extracción)

$p \wedge q$

$\therefore p$

EJEMPLO EN COMPUTACIÓN	APLICACIÓN
El sistema tiene memoria y CPU suficientes. → El sistema tiene memoria suficiente.	Extracción de precondiciones en verificación formal

Resolución (Programación lógica)

$p \vee q$

$\neg p \vee r$

$\therefore q \vee r$

EJEMPLO EN COMPUTACIÓN	APLICACIÓN
El bug está en frontend o en backend. No está en frontend o es crítico. → Está en backend o es crítico.	Prolog, SAT solvers, demostración automática

VERIFICACIÓN DE PROGRAMAS

La lógica de Hoare usa triples $\{P\} C \{Q\}$: si la precondición P se cumple y el comando C ejecuta, entonces la postcondición Q se garantiza. Cada regla es una inferencia lógica.

Triple de Hoare: estructura

$\{ \text{Precondición } P \} \quad \text{Comando } C \quad \{ \text{Postcondición } Q \}$

Esto es una implicación: si P es verdadero antes de C , entonces Q será verdadero después de C .

Regla de asignación (Axioma)	Regla de secuencia (Composición)
$\{ Q[x/E] \} \quad x := E \quad \{ Q \}$	$\{ P \} C1 \{ R \} \quad \{ R \} C2 \{ Q \}$
Ejemplo: $\{ n+1 > 0 \}$ $n := n + 1$ $\{ n > 0 \}$	$\{ P \} C1; C2 \{ Q \}$
Se sustituye x por E en Q .	Encadena instrucciones.



Ejemplo completo: verificación de swap sin variable temporal

1. Precondición:

$\{ a = A \wedge b = B \}$

2. Instrucción 1:

$a := a + b$

$\{ a = A+B \wedge b = B \}$

3. Instrucción 2:

$b := a - b$

$\{ a = A+B \wedge b = A \}$

4. Instrucción 3:

$a := a - b$

$\{ a = B \wedge b = A \}$

5. Postcondición:

$\{ a = B \wedge b = A \}$

Swap correcto.

Aplicación en Inteligencia Artificial

Las reglas de inferencia son la base de los sistemas expertos, la lógica de predicados y la verificación formal de modelos de IA.

Ejemplo 1 — Sistema experto de diagnóstico médico

```
# Base de conocimiento (reglas → hechos)
reglas = [
    "fiebre ∧ tos → posible_gripe",    # Conjunción + MP
    "posible_gripe → recomendar_reposo", # SH
    "¬mejora → consultar_medico"       # MT
]

# Motor de inferencia aplica MP en cadena:
# fiebre=V ∧ tos=V → posible_gripe=V    (MP)
# posible_gripe=V → recomendar_reposo=V  (SH + MP)
```

Ejemplo 2 — Verificación formal de algoritmos (Lógica de Hoare)

Se utiliza la triplete de Hoare: $\{Precondición\}$ Programa $\{Postcondición\}$

```
# Demostración de correctitud con inferencia:
# p:     $n \geq 0$  (precondición)
# q:    factorial(n) devuelve n! (postcondición)
#
# Caso base:  $p(0) \rightarrow q(0)$  verificado directamente
# Paso ind.:  $p(k) \rightarrow q(k) \wedge p(k+1) \rightarrow q(k+1)$ 
# Por SH:  $p(n) \rightarrow q(n)$  para todo  $n \geq 0$  ✓

def factorial(n):
    if n == 0: return 1    # caso base: q(0) verdadero
    return n * factorial(n-1) # paso inductivo
```



Lógica Proposicional

Leyes de las Proposiciones & Reglas de Inferencia

Leyes de las Proposiciones

Proposiciones lógicamente equivalentes ($\Leftrightarrow$) que permiten simplificar y transformar expresiones proposicionales complejas.

LEY DE ÍDEM POTENCIA	LEY DE COMPLEMENTO	LEYES CONMUTATIVAS
$P \vee P \Leftrightarrow P$ $P \wedge P \Leftrightarrow P$ Una proposición combinada consigo misma es siempre equivalente a sí misma. Elimina redundancias.	$\neg F \Leftrightarrow V \quad \neg V \Leftrightarrow F$ $\neg\neg P \Leftrightarrow P$ $P \wedge \neg P \Leftrightarrow F$ $P \vee \neg P \Leftrightarrow V$ La negación invierte valores de verdad. La doble negación se cancela. Una proposición y su negación forman contradicción ($\wedge$) o tautología ($\vee$).	$P \vee Q \Leftrightarrow Q \vee P$ $P \wedge Q \Leftrightarrow Q \wedge P$ El orden de los operandos en conjunción y disyunción no afecta el valor de verdad resultante.

LEYES ASOCIATIVAS	LEYES DE IDENTIDAD	LEYES DISTRIBUTIVAS
$(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$ $(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$ La agrupación de operandos bajo el mismo operador lógico no cambia el resultado.	$P \wedge V \Leftrightarrow P \quad P \wedge F \Leftrightarrow F$ $P \vee F \Leftrightarrow P \quad P \vee V \Leftrightarrow V$ Combinar con V o F produce resultados predecibles: V es absorbente en $\vee$, F es absorbente en $\wedge$.	$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$ $P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$ Permiten distribuir $\wedge$ sobre $\vee$ y viceversa, análogamente a la distribución algebraica.

LEYES DE DE MORGAN	
$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$ Negación de una conjunción $\rightarrow$ disyunción de negaciones	$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$ Negación de una disyunción $\rightarrow$ conjunción de negaciones



Reglas de Inferencia

Esquemas de razonamiento válido: dadas premisas verdaderas, se deduce ($\therefore$) con certeza lógica una conclusión verdadera.

MODUS PONENS $P \rightarrow Q$ P $\therefore Q$ Afirmando el antecedente. Si el condicional es verdadero y el antecedente se cumple, el consecuente necesariamente se cumple.	MODUS TOLLENS $P \rightarrow Q$ $\neg Q$ $\therefore \neg P$ Negando el consecuente. Si el consecuente es falso en un condicional verdadero, el antecedente también debe ser falso.
SILOGISMO HIPOTÉTICO $P \rightarrow Q$ $Q \rightarrow R$ $\therefore P \rightarrow R$ Encadenamiento de implicaciones. La transitividad del condicional: si P lleva a Q y Q lleva a R, entonces P lleva a R.	SILOGISMO DISYUNTIVO $P \vee Q$ $\neg P$ $\therefore Q$ Si una disyunción es verdadera y uno de sus disyuntos es falso, el otro disyunto debe ser necesariamente verdadero.
ADICIÓN P $\therefore P \vee Q$ Si P es verdadera, podemos agregar cualquier proposición Q mediante una disyunción, pues basta que un disyunto sea verdadero.	SIMPLIFICACIÓN $P \wedge Q$ $\therefore P$ De una conjunción verdadera podemos extraer cualquiera de sus componentes, ya que ambos deben ser verdaderos.
CONJUNCIÓN P Q $\therefore P \wedge Q$ Si P y Q son verdaderas de forma independiente, podemos unir las legítimamente en una conjunción.	ABSORCIÓN $P \rightarrow Q$ $\therefore P \rightarrow (P \wedge Q)$ Si P implica Q, entonces P implica la conjunción de sí misma con Q: cuando P es verdadera, Q también lo es.
DILEMA CONSTRUCTIVO $(P \rightarrow Q) \wedge (R \rightarrow S)$ $P \vee R$ $\therefore Q \vee S$ Teniendo dos condicionales y sabiendo que al menos uno de sus antecedentes es verdadero ($P \vee R$), podemos concluir que al menos uno de sus consecuentes es verdadero. Es la forma constructiva del dilema: construimos una disyunción de consecuentes.	DILEMA DESTRUCTIVO $(P \rightarrow Q) \wedge (R \rightarrow S)$ $\neg Q \vee \neg S$ $\therefore \neg P \vee \neg R$ Si se niega al menos uno de los consecuentes ($\neg Q \vee \neg S$), entonces debe negarse al menos uno de los antecedentes. Es la contraparte destructiva: aplica Modus Tollens a ambos condicionales simultáneamente.



Tabla de Símbolos

Símbolo	Nombre	Símbolo	Nombre	Símbolo	Nombre
$\wedge$	Conjunción "Y" lógico	$\vee$	Disyunción "O" lógico	$\neg$	Negación "NO" lógico
$\rightarrow$	Condicional Si... entonces	$\leftrightarrow$	Bicondicional Si y solo si	$\Leftrightarrow$	Equivalencia Lógicamente igual
$\therefore$	Por lo tanto Conclusión	V / F	Verdadero / Falso — Valores de verdad	$\vdash$	Deducción

6. Preguntas de Control

6.1 Aplique las leyes de De Morgan para simplificar la siguiente expresión lógica:

$\neg(p \wedge q) \vee \neg(r \rightarrow s)$. Muestre paso a paso el procedimiento de simplificación y verifique el resultado con una tabla de verdad.

Expresión Inicial: $\neg(p \wedge q) \vee \neg(r \rightarrow s)$

Descripción	Expresión	Ley aplicada
Reescribir el condicional como disyunción	$r \rightarrow s \equiv \neg r \vee s$	<i>Definición del condicional material</i>
Sustituir en la expresión original	$\neg(p \wedge q) \vee \neg(\neg r \vee s)$	<i>Sustitución directa del paso 1</i>
Aplicar De Morgan a $\neg(p \wedge q)$	$\neg p \vee \neg q$	<i>De Morgan</i>
Aplicar De Morgan a $\neg(\neg r \vee s)$	$r \wedge \neg s$	<i>De Morgan</i> <i>Doble negación</i>
Reunir ambos resultados	$(\neg p \vee \neg q) \vee (r \wedge \neg s)$	<i>Expresión simplificada final</i>

Respuesta final: $(\neg p \vee \neg q) \vee (r \wedge \neg s)$



unl

Universidad
Nacional
de Loja

Verificación Mediante tabla de verdad

p	q	r	s	$\neg p$	$\neg q$	$\neg s$	$p \wedge q$	$r \rightarrow s$	$\neg(p \wedge q) \vee \neg(r \rightarrow s)$	$(\neg p \vee \neg q) \vee (r \wedge \neg s)$
V	V	V	V	F	F	F	V	V	F	F
V	V	V	F	F	F	V	V	F	V	V
V	V	F	V	F	F	F	V	V	F	F
V	V	F	F	F	F	V	V	V	F	F
V	F	V	V	F	V	F	F	V	V	V
V	F	V	F	F	V	V	F	F	V	V
V	F	F	V	F	V	F	F	V	V	V
V	F	F	F	F	V	V	F	V	V	V
F	V	V	V	V	F	F	F	V	V	V
F	V	V	F	V	F	V	F	F	V	V
F	V	F	V	V	F	F	F	V	V	V
F	V	F	F	V	F	V	F	V	V	V
F	F	V	V	V	V	F	F	V	V	V
F	F	V	F	V	V	V	F	F	V	V
F	F	F	V	V	V	F	F	V	V	V
F	F	F	F	V	V	V	F	V	V	V

Las 16 filas de ambas columnas coinciden — equivalencia lógica verificada.



7. Conclusiones

Se concluye que:

- Las reglas de inferencia y las leyes lógicas constituyen el fundamento formal de la computación confiable. Modus Ponens, Modus Tollens, Silogismo Hipotético y las Leyes de De Morgan no son meros ejercicios académicos; son herramientas operativas que permiten verificar programas (lógica de Hoare), construir sistemas expertos, depurar errores y diseñar arquitecturas de software con propiedades demostrables.
- Existe un puente directo entre la teoría matemática y el desarrollo de software: los **conectores lógicos**. En programación, estos se traducen en instrucciones booleanas que permiten al código tomar decisiones basadas en reglas formales de razonamiento.

8. Recomendaciones

Se recomienda que:

- Se sugiere automatizar la comprobación de precondiciones, postcondiciones e invariantes para módulos críticos (cálculos financieros, control de hardware, autenticación).
- La experiencia en depuración empírica es insuficiente para evitar errores sutiles (como inversión de condicionales o uso incorrecto de De Morgan). Incorporar en los cursos de algoritmos y arquitectura de software la práctica sistemática de reglas de inferencia mejora la calidad del código y la capacidad de refactorización segura.



unl

Universidad
Nacional
de Loja

9. Reflexión crítica

El material presentado resulta **sumamente interesante y valioso** porque conecta de manera clara y didáctica los fundamentos de la lógica formal con aplicaciones concretas en computación, inteligencia artificial y verificación de programas. La forma en que se ejemplifica cada regla de inferencia con situaciones reales (errores en frontend/backend, pipelines de datos, diagnósticos médicos) hace que conceptos abstractos cobren sentido práctico de inmediato..

10. Bibliografía / Referencias

[1] O. Levin, Discrete Mathematics: An Open Introduction, 4th ed. University of Northern Colorado, 2022. [En línea]. Disponible en: <https://discrete.openmathbooks.org>

11. Anexos

Matías Ortega

ADP 4

Resultados

► **REGLAS DE INFERENCIA**
Las reglas de inferencia permiten derivar conclusiones válidas a partir de premisas. Son la base del razonamiento deductivo en lógica y en la verificación de programas

→ **MODUS PONENS (Modus ponendo)**

$$\begin{array}{l} p \rightarrow q \\ p \\ \hline \therefore q \end{array}$$

EJEMPLO EN COMPUTACIÓN
Si el compilador no reporta errores, el programa es correcto.
El compilador no reporta errores.
→ El programa es correcto.

APLICACIÓN
Motores de inferencia, sistemas expertos, verificación.

→ **MODUS TOLLENS (Prueba por contradicción)**

$$\begin{array}{l} p \rightarrow q \\ \neg q \\ \hline \therefore \neg p \end{array}$$

EJEMPLO EN COMPUTACIÓN
Si hay un bug, los tests fallan.
Los tests no fallan.
→ No hay bugs

APLICACIÓN
Depuración lógica, refutación de hipótesis

→ **SIOLOGISMO HIPOTÉTICO (Encadenamiento)**

$$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore p \rightarrow r \end{array}$$

EJEMPLO EN COMPUTACIÓN
Si input es válido → se procesa.
Si se procesa → se guarda.
→ Si input válido → se guarda.

APLICACIÓN
Pipelines, flujos de datos, arquitectura por capas

→ ADICIÓN CAMPLIACIÓN

$$\frac{p}{\therefore p \vee q}$$

EJEMPLO EN COMPUTACIÓN

El test pasó.
→ El test pasó o hay un error en otro módulo

APLICACIÓN

Sistemas de diagnóstico con alternativas

→ SIMPLIFICACIÓN (Extracción)

$$\frac{p \wedge q}{\therefore p}$$

EJEMPLO EN COMPUTACIÓN

El sistema tiene memoria y CPU suficientes.
→ El sistema tiene memoria suficiente

APLICACIÓN

Extracción de precondiciones en verificación formal

→ RESOLUCIÓN (Programación lógica)

$$\frac{\frac{p \vee q}{\therefore p \vee r}}{\therefore q \vee r}$$

EJEMPLO EN COMPUTACIÓN

El bug está en frontend o en backend.
No está en frontend o es crítico.
→ Está en backend o es crítico

APLICACIÓN

Prolog, SAT solvers, demostración automática.

► VERIFICACIÓN DE PROGRAMAS

La lógica de Hoare usa triples $\{P\} C \{Q\}$; si la precondición P se cumple y el comando C ejecuta, entonces la postcondición Q se garantiza. Cada regla es una inferencia lógica.

Triple de Hoare: estructura

$\{ \text{Precondición } P \} \text{ Comando } C \{ \text{Postcondición } Q \}$

Esto es una implicación: si P es verdadero antes de C , entonces Q será verdadero después de C .

Regla de asignación (Axioma)

$$\{ Q[x/E] \} x := E \{ Q \}$$

Ejemplo:

Regla de secuencia (composición)

$$\frac{\{ P \} C_1 \{ R \} \quad \{ R \} C_2 \{ Q \}}{\{ P \} C_1 ; C_2 \{ Q \}}$$

$$\{ P \} C_1 ; C_2 \{ Q \}$$

Encadenar instrucciones


```
{ n > 0 }
n := n + 1
{ n > 0 }
```

se sustituye x por x en a

Ejemplo completo: Verificación de swap sin variable temporal

1. Precondición:
 $\{ a = A \wedge b = B \}$

2. Instrucción 1:
 $a := a + b$
 $\{ a = A + B \wedge b = B \}$

3. Instrucción 2:
 $b := a - b$
 $\{ a = A + B \wedge b = A \}$

4. Instrucción 3:

$a := a - b$
 $\{ a = B \wedge b = A \}$

5. Postcondición:
 $\{ a = B \wedge b = A \}$
swap correcto

► Aplicación en Inteligencia Artificial

Las reglas de inferencia son la base de datos de los sistemas expertos
la lógica de predicados y la verificación formal de modelos de IA

Ejemplo 1 - Sistema experto de diagnóstico médico

Base de conocimientos (reglas $\rightarrow$ hechos)

Reglas = $\{$
"fiebre $\wedge$ tos $\rightarrow$ posible gripe", # Conjunción + MP
"posible gripe $\rightarrow$ recomendar reposo", # SH
"no mejora $\rightarrow$ consultar médico" # MT
 $\}$

Motor de inferencia aplica MP en cadena.

"fiebre = V $\wedge$ tos = V $\Rightarrow$ posible gripe = V" (MP)

"posible gripe = V $\Rightarrow$ recomendar reposo = V" (SH + MP)

Ejemplo 2 - Verificación formal de algoritmos (Lógica de Hoare)

se utiliza la tripleta de Hoare: $\{ \text{Precondición} \} \text{Programa} \{ \text{Postcondición} \}$

Demostración de corrección por inferencia:

p: $n \geq 0$ (precondición)

q: factorial(n) devuelve n! (postcondición)

caso base: $p(0) \Rightarrow q(0)$ verificado directamente

caso ind: $p(k) \Rightarrow q(k) \wedge p(k+1) \Rightarrow q(k+1)$

por SH: $q(n) \Rightarrow q(n)$ para todo $n \geq 0$

def factorial(n)

if $n == 0$: return 1

return n * factorial(n-1)

caso base: $q(0)$ verdadero

paso inductivo

LOGICA MATEMATICA - RESUMEN VISUAL

LOGICA PROPOSICIONAL

Reges de las Proposiciones & Reglas de Inferencia

Reges de las Proposiciones

Proposiciones lógicamente equivalentes ($\Leftrightarrow$) que permiten simplificar y transformar expresiones proposicionales complejas.

Ley de Idempotencia

$$P \vee P \Leftrightarrow P$$

$$P \wedge P \Leftrightarrow P$$

Elimina redundancias

Ley de Complemento

$$F \Leftrightarrow \neg V, \neg V \Leftrightarrow F$$

$$\neg P \Leftrightarrow \neg P$$

$$P \vee \neg P \Leftrightarrow V$$

2ª doble negación se cancela

Reges Conmutativas

$$P \vee Q \Leftrightarrow Q \vee P$$

$$P \wedge Q \Leftrightarrow Q \wedge P$$

El orden de los operandos no afecta el valor de verdad.

Reges Asociativas

$$(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$$

$$(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$$

2ª agrupación de operandos bajo el mismo operador lógico no cambia el resultado

Reges de Identidad

$$P \wedge V \Leftrightarrow P, P \wedge F \Leftrightarrow F$$

$$P \vee F \Leftrightarrow P, P \vee V \Leftrightarrow V$$

Combinar con V o F para dar resultados predecibles. V es absorbente en $\vee$, F es absorbente en $\wedge$.

Reges distributivas

$$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$$

$$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$$

Permiten distribuir $\wedge$ sobre $\vee$ y viceversa, análogamente en distribución algebraica.

REGES DE MORGAN

$$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$$

Negación de una conjunción
→ disyunción de negaciones

$$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$$

Negación de una disyunción
→ conjunción de negaciones

REGES DE INFERENCIA

Esquemas de razonamiento válidos, dadas premisas verdaderas, se deduce (:) con certeza lógica una conclusión verdadera.

MODUS PONENS

$$P \rightarrow Q$$

$$P$$

$$\therefore Q$$

Afirmando el antecedente. Si el condicional es verdadero y el antecedente se cumple, el consecuente necesariamente se cumple.

MODUS TOLLENS

$$P \rightarrow Q$$

$$\neg Q$$

$$\therefore \neg P$$

Negando el consecuente. Si el consecuente es falso en un condicional verdadero, el antecedente también debe ser falso.

SÍLOGISMO HIPOTÉTICO

$$\begin{array}{l} P \rightarrow Q \\ Q \rightarrow R \end{array}$$

$$\therefore P \rightarrow R$$

Encadenamiento de implicaciones. 2a transitividad del condicional: si P lleva a Q y Q lleva a R, entonces P lleva a R.

ADICIÓN

$$\begin{array}{l} P \\ \hline \end{array}$$

$$\therefore P \vee Q$$

Si P es verdadera, podemos agregar cualquier proposición Q mediante una disyunción, pues basta que un disyunto sea verdadero.

CONJUNCIÓN

$$\begin{array}{l} P \\ Q \end{array}$$

$$\therefore P \wedge Q$$

Si P y Q son verdaderas de forma independiente, podemos unirlos legítimamente en una conjunción.

DILEMA CONSTRUCTIVO

$$(P \rightarrow Q) \wedge (R \rightarrow S)$$

$$\begin{array}{l} P \vee R \end{array}$$

$$\therefore Q \vee S$$

Teniendo dos condicionales y sabiendo que al menos uno de sus antecedentes es verdadero (PVR), podemos concluir que al menos uno de sus consecuentes es verdadero.

SÍLOGISMO DISYUNTIVO

$$\begin{array}{l} P \vee Q \\ \neg P \end{array}$$

$$\therefore Q$$

Si una disyunción es verdadera y uno de sus disyuntos es falso, el otro disyunto debe ser necesariamente verdadero.

SIMPLIFICACIÓN

$$\begin{array}{l} P \wedge Q \\ \hline \end{array}$$

$$\therefore P$$

De una conjunción verdadera podemos extraer cualquiera de sus componentes, ya que ambos deben ser verdaderos.

ABSORCIÓN

$$\begin{array}{l} P \rightarrow Q \\ \hline \end{array}$$

$$\therefore P \rightarrow (P \wedge Q)$$

Si P implica Q, entonces P implica la conjunción de sí misma con Q cuando P es verdadero, Q también lo es.

DILEMA DESTRUCTIVO

$$(P \rightarrow Q) \wedge (R \rightarrow S)$$

$$\neg Q \vee \neg S$$

$$\therefore \neg P \vee \neg R$$

Si se niega al menos uno de los consecuentes ($\neg Q \vee \neg S$), entonces debe negarse al menos uno de los antecedentes.



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Erick Andrade

DIA	MES	AÑO	NOTA

Anexos del APEU

Reglas de inferencia

Las reglas de inferencia permiten derivar conclusiones válidas a partir de premisas. Son la base del razonamiento deductivo en lógica y en la verificación de programas.

Modus Ponens

$$P \rightarrow q$$

$$\frac{P}{q}$$

Ejemplo en computación

Si el compilador no reporta errores, el programa es correcto. El compilador no reporta errores. $\therefore$ El programa es correcto.

Aplicación

Motores de inferencia, sistemas expertos, verificación.

Modus Tollens (Pruebas por contradicción)

$$p \rightarrow q$$

$$\frac{\neg q}{\neg p}$$

Ejemplo en computación

Si hay bug, los tests fallan.
Los tests no fallan.
 $\rightarrow$ No hay bug.

Aplicación

Depuración lógica, refutación de hipótesis

Silogues hipotética (Fundamento)

$$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline p \rightarrow r \end{array}$$

Ejemplo en computación

Si el input es válido $\rightarrow$ se procesa.

Si se procesa $\rightarrow$ se guarda.

$\rightarrow$ Si input válido $\rightarrow$ se guarda.

Aplicación

Pipelines, flujos de datos, arquitectura por copias.

Adición (Ampliación)

$$\begin{array}{l} p \\ p \vee q \end{array}$$

Ejemplo en computación

El test pasó.

$\rightarrow$ El test pasó o hay un error en otro módulo.

Aplicación

Sistemas de diagnóstico con alternativas.

Simplificación (Reducción)

$$\begin{array}{l} p \wedge q \\ p \end{array}$$

Ejemplo de computación

El sistema tiene memoria y CPU suficiente.

$\rightarrow$ El sistema tiene memoria suficiente.

Aplicación

Extracción de precondiciones en verificación formal.

LOGICA proposicional

Leyes de las proposiciones & Reglas de inferencia

Leyes de las proposiciones

Proposiciones lógicamente equivalentes ($\Leftrightarrow$) que permiten simplificar y transformar expresiones proposicionales complejas.

<u>Ley de Idempotencia</u>	<u>Ley de complemento</u>	<u>Leyes constantes</u>
$P \vee P \Leftrightarrow P$	$\neg(\neg P) \Leftrightarrow P$	$P \vee Q \Leftrightarrow Q \vee P$
$P \wedge P \Leftrightarrow P$	$P \wedge \neg P \Leftrightarrow F$	$P \wedge Q \Leftrightarrow Q \wedge P$
<u>Eliminación redundancia</u>	$P \vee \neg P \Leftrightarrow V$	El orden de los operandos no
	La doble negación se cancela	afecta el valor de verdad resultante

<u>Leyes asociativas</u>	<u>Ley de absorción</u>	<u>Leyes distributivas</u>
$(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$	$P \wedge V \Leftrightarrow P$	$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$
$(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$	$P \vee F \Leftrightarrow P$	$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$

La agrupación de operandos bajo el mismo operador lógico no cambia el resultado. Combinar con $\vee$ o $\wedge$ produce resultados predecibles. Permite distribuir $\wedge$ sobre $\vee$ y $\vee$ sobre $\wedge$, análogamente a la distribución algebraica.

Leyes de Morgan

$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$	$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$
Negación de una conjunción	Negación de una disyunción
$\rightarrow$ disyunción de negaciones	$\rightarrow$ conjunción de negaciones

Resolución Programación lógicas

$p \vee q$

$\neg p \vee r$

$q \vee r$

Ejemplo en computación

El bug está en frontend o en backend.

No está en frontend o es crítico.

→ Está en backend o es crítico

Aplicación

Prolog, SAT solvers, demostración

automática

Verificación de programas

La lógica de Hoare usa triples $\{P\} C \{Q\}$ si la precondición P se cumple y el comando C ejecuta, entonces la postcondición Q se garantiza. Cada regla es una inferencia lógica.

Triple de Hoare estructura

$\{Precondición P\}$ Comando C $\{Postcondición Q\}$

Esto es una implicación: si P es verdadero antes de C , entonces Q será verdadero después de C .

Regla de asignación (Axioma)

$\{Q \wedge E\} x := E \{Q\}$

Ejemplo:

$\{n \neq 0\}$

$n := n + 1$

$\{n > 0\}$

Regla de secuencia (Composición)

$\{P\} C_1 \{R\} \quad \{R\} C_2 \{Q\}$

$\{P\} C_1 ; C_2 \{Q\}$

Encadenar instrucciones.

Ejemplo completo verificación de swap en variable temporal

1. Precondición:

$$\{a = A \wedge b = B\}$$

2. Instrucción 1:

$$a := a + b$$

$$\{a = A + B \wedge b = B\}$$

3. Instrucción 2:

$$b := a - b$$

$$\{a = A + B \wedge b = A\}$$

4. Instrucción 3:

$$a := a - b$$

$$\{a = B \wedge b = A\}$$

5. Postcondición:

$$\{a = B \wedge b = A\}$$

Swap correcto.

Aplicación en Inteligencia Artificial

Las reglas de inferencia son la base de los sistemas expertos, la lógica de predicados y la verificación formal de modelos de IA.

Ejemplo 1 - Sistema experto de diagnóstico médico

* Base de conocimiento (reglas $\rightarrow$ hechos)

reglas = [

"fiebre $\wedge$ tos $\rightarrow$ posible_gripe", * Conjunción + MP

"posible_gripe $\rightarrow$ recomendar_reposo", * SH

"sintomas $\rightarrow$ consultar_medico" * MT

]

* Motor de inferencia aplica MP en cadena:

* fiebre = V $\wedge$ tos = V $\rightarrow$ posible_gripe = V (MP)

* posible_gripe = V $\rightarrow$ recomendar_reposo = V (SH + MP)

Ejemplo 2 - Verificación formal de algoritmos (Lógica de Hoare)

Se utiliza la tripleta de Hoare: {Precondición} Programa {Postcondición}

* Demostración de correctitud con inducción:

* p : $n \geq 0$ (precondición)

* q : $\text{factorial}(n)$ devuelve $n!$ (postcondición)

*
*

* (caso base: $p(0) \rightarrow q(0)$) verificando directamente

* Paso ind: $p(k) \rightarrow q(k) \wedge p(k+1) \rightarrow q(k+1)$

* Por SH: $p(n) \rightarrow q(n)$ para todo $n \geq 0$ ✓

def $\text{factorial}(n)$:

if $n == 0$: return 1 * caso base: $q(0)$ verdadero

return $n * \text{factorial}(n-1)$ * paso inductivo



Reglas de Inferencia

Esquemas de razonamiento válido: dadas premisas verdaderas, se deduce ($\therefore$) con certeza lógica una conclusión verdadera.

Modus ponens

$$\begin{array}{l} P \rightarrow Q \\ P \\ \hline \therefore Q \end{array}$$

Afirmando el antecedente. Si el condicional es verdadera y el antecedente se cumple, el consecuente necesariamente se cumple.

Modus tollens

$$\begin{array}{l} P \rightarrow Q \\ \neg Q \\ \hline \therefore \neg P \end{array}$$

Negando el consecuente. Si el consecuente es falso en un condicional verdadera, el antecedente también debe ser falso.

Silogismo hipotético

$$\begin{array}{l} P \rightarrow Q \\ Q \rightarrow R \\ \hline \therefore P \rightarrow R \end{array}$$

Encadenamiento de implicaciones. La transitividad del condicional: Si P lleva a Q y Q lleva a R, entonces P lleva a R.

Silogismo disyuntivo

$$\begin{array}{l} P \vee Q \\ \neg P \\ \hline \therefore Q \end{array}$$

Si una disyunción es verdadera y uno de sus disyuntos es falso, el otro disyunto debe ser necesariamente verdadera.

Adición

$$\begin{array}{l} P \\ \hline \therefore P \vee Q \end{array}$$

Si P es verdadera, podemos agregar cualquier proposición Q mediante una disyunción, pues basta de un disyuntivo sea verdadero.

Simplificación

$$\begin{array}{l} P \wedge Q \\ \hline \therefore P \end{array}$$

De una conjunción verdadera podemos extraer cualquiera de sus componentes, ya que ambos deben ser verdaderas.

DIA: MES: AÑO:	NOTA: /
<u>Conjunción</u> $\begin{array}{l} P \\ Q \\ \hline P \wedge Q \end{array}$	<u>Abstracción</u> $\begin{array}{l} P \rightarrow Q \\ \hline P \rightarrow (P \wedge Q) \end{array}$
Si P y Q son verdaderas de forma independiente, podemos unirlos legítimamente en una conjunción.	Si P implica Q, entonces P implica la conjunción de sí misma con Q: cuando P es verdadera, Q también lo es.
<u>Dilema constructiva</u> $\begin{array}{l} (P \rightarrow Q) \wedge (R \rightarrow S) \\ P \vee R \\ \hline Q \vee S \end{array}$	<u>Dilema destructiva</u> $\begin{array}{l} (P \rightarrow Q) \wedge (R \rightarrow S) \\ \neg Q \vee \neg S \\ \hline \neg P \vee \neg R \end{array}$
Teniendo dos condicionales y sabiendo que al menos uno de sus antecedentes es verdadero ($P \vee R$), podemos concluir que al menos uno de sus consecuentes es verdadero.	Si se niega al menos uno de los consecuentes ($\neg Q \vee \neg S$), entonces debe negarse al menos uno de los antecedentes.

Johan Rojas

ACE 1 : FASE 4 : Matemáticas Discretas

Resultados

Reglas de inferencia

Las reglas de inferencia permiten derivar conclusiones válidas a partir de premisas. Son la base del razonamiento deductivo en lógica y en la verificación de programas.

Modus Ponens

$$\begin{array}{l} p \rightarrow q \\ p \\ \hline \therefore q \end{array}$$

Ejemplo en computación	Aplicación
- Si el compilador no reporta errores, el programa es correcto. - El compilador no reporta errores. $\therefore$ El programa es correcto.	Motores de eficiencia, sistemas expertos, verificación.

Modus Tollens (prueba por contradicción)

$$\begin{array}{l} p \rightarrow q \\ \neg q \\ \hline \therefore \neg p \end{array}$$

Ejemplo en computación	Aplicación
Si hay un bug, los test fallan. Los test no fallan. $\therefore$ No hay bug.	Depuración lógica, refutación lógica.

Silogismo hipotético (encadenamiento)

$$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore p \rightarrow r \end{array}$$

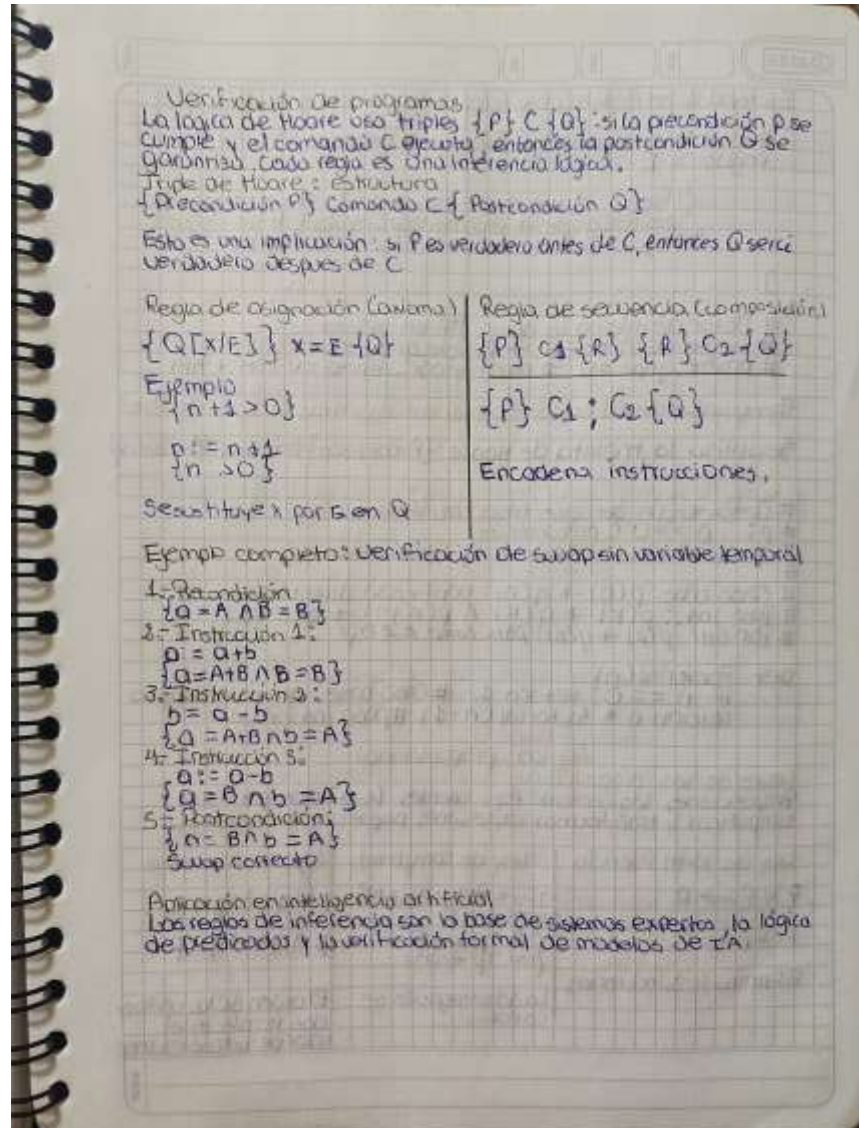


unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Ejemplo de computación Si input es válido $\rightarrow$ se procesa. Si se procesa $\rightarrow$ se guarda $\rightarrow$ Si input es válido $\rightarrow$ se guarda Adición P $\therefore P \vee Q$	Aplicación Pipelines, flujo de datos, arquitectura por copias.
Ejemplo de computación El test pasó $\rightarrow$ El test pasó o hay error en otro módulo Simplificación (Extracción) $P \quad N \quad Q$ $\therefore P$	Aplicación Sistemas de diagnóstico con alternativas.
Ejemplo de computación El sistema tiene memoria y CPU suficientes. $\rightarrow$ El sistema tiene memoria suficiente Resolución (programación lógica) $P \vee Q$ $\neg P \vee R$ $\therefore Q \vee R$	Aplicación Extracción de precondiciones en verificación formal.
Ejemplo de computación El bug está en frontend o en backend NO está en frontend Describiendo $\rightarrow$ Está en backend Describiendo	Aplicación Prolog, SAT solvers, demostración automática.



Ejemplo 1 - Sistema experto de diagnóstico médico

Base de conocimientos (reglas $\rightarrow$ hechos)

reglas = [

"Fiebre $\wedge$ tos $\rightarrow$ posible_gripe" #conjunción + MP
"posible_gripe $\rightarrow$ recomendar_reposo" #SH
"mejora $\rightarrow$ consultar_médico" #MT

]

Motor de inferencia aplica MP en cadena:

Fiebre = V $\wedge$ tos = V $\rightarrow$ posible_gripe = V (MP)

posible_gripe = V $\rightarrow$ recomendar_reposo = V (SH + MP)

Ejemplo 2 - Verificación formal de algoritmo (Lógica de Hoare)

Se utiliza la tripleta de Hoare: {Precondición} Programa {Postcondición}

Demostración de correctitud con inferencia:

p: $n \geq 0$ (precondición)

q: factorial(n) devuelve n! (postcondición)

#

Caso base: $p(0) \rightarrow q(0)$ verificado directamente.

Paso ind.: $p(k) \rightarrow q(k) \wedge p(k+1) \rightarrow q(k+1)$

Por SH: $p(n) \rightarrow q(n)$ para todo $n \geq 0$ ✓

def factorial(n):

if $n == 0$: return 1 # caso base: $q(0)$ verdadero

return n * factorial(n-1) # paso inductivo

Lógica proposicional

Leyes de las proposiciones

Proposiciones lógicamente equivalentes ($\leftrightarrow$) que permiten simplificar y transformar expresiones proposicionales complejas

Ley de Idempotencia

$p \vee p \leftrightarrow p$

$p \wedge p \leftrightarrow p$

Elimina redundancias

Ley de complemento

$\neg F \leftrightarrow V$ $\neg V \leftrightarrow F$

$\neg \neg p \leftrightarrow p$

$p \wedge \neg p \leftrightarrow F$

$p \vee \neg p \leftrightarrow V$

La doble negación se cancela

Leyes de conmutación

$p \vee q \leftrightarrow q \vee p$

$p \wedge q \leftrightarrow q \wedge p$

El orden de los operandos no afecta el valor de verdad resultante.



Leyes asociativas	Leyes de identidad	Leyes distributivas
$(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$ $(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$	$P \wedge V \Leftrightarrow P$ $P \vee V \Leftrightarrow V$ $P \wedge F \Leftrightarrow F$ $P \vee F \Leftrightarrow P$	$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$ $P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$
La operación de operandos bajo el mismo operador lógico no cambia el resultado.	V es absorbente en $\vee$. F es absorbente en $\wedge$.	Permite distribuir $\wedge$ sobre $\vee$ y viceversa.

Leyes de Morgan

$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$ Negación de una conjunción + disyunción de negaciones	$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$ Negación de una disyunción + conjunción de negaciones
---	---

Reglas de inferencia

Esquemas de razonamiento válido: dadas premisas verdaderas, se deduce (...) con certeza lógica una conclusión verdadera.

Modus Ponens $\begin{array}{l} p \rightarrow q \\ p \\ \hline \therefore q \end{array}$ Afirmando el antecedente. Si el condicional es verdadero y el antecedente se cumple.	Modus Tollens $\begin{array}{l} p \rightarrow q \\ \neg q \\ \hline \therefore \neg p \end{array}$ Negando el consecuente. Si el consecuente es falso en un condicional verdadero el antecedente también debe ser falso.
Silogismo hipotético $\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore p \rightarrow r \end{array}$ Si p lleva a q y q lleva a r, entonces p lleva a r.	Silogismo disyuntivo $\begin{array}{l} p \vee q \\ \neg p \\ \hline \therefore q \end{array}$ Si una disyunción es verdadera y uno de sus disyuntos es falso, el otro disyunto debe ser necesariamente verdadero.



Adición	Simplificación
$\frac{p}{\therefore p \vee q}$ Si p es verdadera, podemos agregar cualquier proposición q mediante una disyunción.	$\frac{p \wedge q}{\therefore p}$ De una conjunción verdadera podemos extraer cualquiera de sus componentes.
$\frac{p}{q}$ $\therefore p \wedge q$ Si p y q son verdaderas independientemente, podemos usarlas en una conjunción.	$\frac{p \rightarrow q}{\therefore p \rightarrow (p \wedge q)}$ Si p implica q, entonces p implica la conjunción de sí misma con q cuando p es V, q también.
$\frac{(p \rightarrow q) \wedge (r \rightarrow s)}{p \vee r}$ $\therefore q \vee s$ Teniendo condicionales y sabiendo que al menos uno de sus antecedentes es verdadero, podemos concluir que al menos uno de sus consecuentes es verdadera.	$\frac{(p \rightarrow q) \wedge (r \rightarrow s)}{\neg p \vee \neg s}$ $\therefore \neg p \vee \neg r$ Si se niega al menos uno de los consecuentes, entonces, debe negarse al menos uno de los antecedentes.



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Tyrone Encarnación

DÍA	MES	AÑO	NOTA

APE 4

Resultados

Reglas de Inferencia

Modus Ponens (Más usada)

$$\begin{array}{l}
 p \rightarrow q \\
 p \\
 \hline
 \therefore q
 \end{array}$$

Ejemplo en computación

Si el compilador no reporta errores, el programa es correcto.

El compilador no reporta errores
→ El programa es correcto

Modus Tollens (Prueba por contradicción)

$$\begin{array}{l}
 p \rightarrow q \\
 \neg q \\
 \hline
 \therefore \neg p
 \end{array}$$

Ejemplo en computación

Si hay un bug, los tests fallan.
Los tests no fallan.
→ No hay bug

Silogismo hipotético (Encadenamiento)

$$\begin{array}{l}
 p \rightarrow q \\
 q \rightarrow r \\
 \hline
 \therefore p \rightarrow r
 \end{array}$$

Ejemplo en computación

Si input es válido → se procesa.
Si se procesa → se guarda.
→ Si input válida → se guarda

Aplicación

Motores de inferencia, sistemas expertos, verificación

Aplicación

Depuración lógica, refutación hipotética

Aplicación

Si p entonces, flujo de datos arquitecturas por capas

Adición (Ampliación)

$$\frac{p}{\therefore p \vee q}$$

Ejemplo de computación

1 test pasó
El test pasó o hay un error en otro módulo.

Aplicación

Sistemas de diagnóstico con alternativas.

Simplificación (Extracción)

$$\frac{p \wedge q}{\therefore p}$$

Ejemplo en computación

El sistema tiene memoria y CPU suficiente.

→ El sistema tiene memoria suficiente.

Aplicación

Extracción de precondiciones en verificación formal.

Resolución (Programación lógica)

$$\frac{p \vee q \quad \neg p \vee r}{\therefore q \vee r}$$

Ejemplo en computación

El bug está en frontend o en backend.

No está en frontend o es crítico.

→ Está en backend o es crítico.

Aplicación

Prolog, SAT solvers, demostración automática.

Verificación de programas

La lógica de Hoare usa triplas $\{P\} C \{Q\}$: si la precondición P se cumple y el comando C ejecuta, entonces la postcondición Q se garantiza. Cada regla es una inferencia lógica.

Triple de Hoare: estructura

Precondición P Comando C Postcondición Q

DIA MES AÑO
 NITSA

Esto es una implicación; si P es verdadero antes de C , entonces Q será verdadero después de C .

Regla de asignación (Axioma)

$\{Q\} [x/E] \{x := E\} \{Q\}$

Ejemplo:

$\{n+1 > 0\}$

$n := n + 1$

$\{n > 0\}$

Se sustituye x por E en Q

Ejemplo completo: verificación de swap sin variable temporal

1. Precondición

$\{a = A \wedge b = B\}$

2. Instrucción 1:

$a := a + b$

$\{a = A + B \wedge b = B\}$

3. Instrucción 2:

$b := a - b$

$\{a = A + B \wedge b = A\}$

4. Instrucción 3:

$a := a - b$

$\{a = B \wedge b = A\}$

5. Postcondición

$\{a = B \wedge b = A\}$

Swap correcto

Regla de secuencia (Composición)

$\{P\} C_1 \{R\} \quad \{R\} C_2 \{Q\}$

$\{P\} C_1 ; C_2 \{Q\}$

Encadena instrucciones

MES: AÑO: NOTA: /

Aplicación en Inteligencia Artificial

Los reglas de inferencia son la base de los sistemas expertos, la lógica de predicados y la verificación formal de los de IA.

Ejemplo 1 - Sistema experto de diagnóstico médico

Base de conocimiento (reglas $\rightarrow$ hechos)

reglas = [

"fiebre $\wedge$ tos $\Rightarrow$ posible_gripe", #Conjunción + MP

"posible_gripe $\Rightarrow$ recomendar_reposo", #SH

" $\neg$ mejora $\Rightarrow$ consultar_médico" #MT

]

Motor de inferencia aplica MP en cadena:

fiebre = V $\wedge$ tos = V $\rightarrow$ posible_gripe = V (MP)

posible_gripe = V $\Rightarrow$ recomendar_reposo = V (SH + MP)

Ejemplo 2 - Verificación formal de algoritmos (Lógica de Hoare)

Se utiliza la tripleta de Hoare: {Precondición} Programa {Postcondición}

Demostración de correctitud con inferencia:

p: $n \geq 0$ (precondición)

q: factorial(n) devuelve $n!$ (postcondición)

#

Caso base: $p(0) \rightarrow q(0)$ verificado directamente

Paso ind.: $p(k) \rightarrow q(k) \wedge p(k+1) \rightarrow q(k+1)$

Por SH: $p(n) \rightarrow q(n)$ para todo $n \geq 0$ ✓

def factorial(n):

if $n == 0$: return 1 # caso base: $q(0)$ verdadera

return $n * \text{factorial}(n-1)$ # paso inductivo



DÍA MES AÑO NOTA

Lógica matemática - Resumen Visual

Lógica Proposicional

Leyes de las Proposiciones

Ley de Idem. Potencia

$$P \vee P \Leftrightarrow P$$

$$P \wedge P \Leftrightarrow P$$

Elimina redundancias

Leyes conmutativas

$$P \vee Q \Leftrightarrow Q \vee P$$

$$P \wedge Q \Leftrightarrow Q \wedge P$$

El orden de operandos no afecta el valor de verdad resultante.

Leyes de Identidad

$$P \wedge V \Leftrightarrow P \quad P \wedge F \Leftrightarrow F$$

$$P \vee F \Leftrightarrow P \quad P \vee V \Leftrightarrow V$$

Combinar con V o F produce resultados predecibles: V es absorbente en $\vee$, F es absorbente en $\wedge$.

Leyes de

Morgan

$$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$$

Negación de una conjunción

→ disyunción de negaciones

Ley de complemento

$$\neg F \Leftrightarrow V \quad \neg V \Leftrightarrow F$$

$$\neg \neg P \Leftrightarrow P$$

$$P \wedge \neg P \Leftrightarrow F$$

$$P \vee \neg P \Leftrightarrow V$$

La doble negación se cancela

Leyes asociativas

$$(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$$

$$(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$$

La agrupación de operandos bajo el mismo operador lógico no cambia el resultado.

Leyes Distributivas

$$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$$

$$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$$

Permiten distribuir $\wedge$ sobre $\vee$ y viceversa, análogamente a la distribución algebraica.

$$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$$

Negación de una disyunción

→ conjunción de negaciones

DIA MES AÑO
 NOTA

Reglas de Inferencia

Modus Ponens

$$\begin{array}{l}
 P \rightarrow Q \\
 P \\
 \hline
 \therefore Q
 \end{array}$$

Si el condicional es verdadero y el antecedente se cumple, el consecuente necesariamente se cumple.

Silogismo hipotético

$$\begin{array}{l}
 P \rightarrow Q \\
 Q \rightarrow R \\
 \hline
 \therefore P \rightarrow R
 \end{array}$$

Encadenamiento de implicaciones: La transitividad del condicional: si P lleva a Q y Q lleva a R, entonces P lleva a R.

Adición

$$\begin{array}{l}
 P \\
 \hline
 \therefore P \vee Q
 \end{array}$$

Si P es verdadera, podemos agregar cualquier proposición Q mediante una disyunción, pues basta que un disyunto sea verdadero.

Conjunción

$$\begin{array}{l}
 P \\
 Q \\
 \hline
 \therefore P \wedge Q
 \end{array}$$

Si P y Q son verdaderas de forma independiente, podemos unirlos legítimamente en una conjunción.

Modus Tollens

$$\begin{array}{l}
 P \rightarrow Q \\
 \neg Q \\
 \hline
 \therefore \neg P
 \end{array}$$

Negando el consecuente. Si el consecuente es falso en un condicional verdadero, el antecedente también debe ser falso.

Silogismo Disyuntivo

$$\begin{array}{l}
 P \vee Q \\
 \neg P \\
 \hline
 \therefore Q
 \end{array}$$

Si una disyunción es verdadera y uno de sus disyuntos es falso, el otro disyunto debe ser necesariamente verdadero.

Simplificación

$$\begin{array}{l}
 P \wedge Q \\
 \hline
 \therefore P
 \end{array}$$

De una conjunción verdadera podemos extraer cualquiera de sus componentes, ya que ambos deben ser verdaderos.

Absorción

$$\begin{array}{l}
 P \rightarrow Q \\
 \hline
 \therefore P \rightarrow (P \wedge Q)
 \end{array}$$

Si P implica Q, entonces P implica la conjunción de sí misma con Q, cuando P es verdadero, Q también lo es.



DIA MES ANO

Dilema Destructivo

$$(P \rightarrow Q) \wedge (R \rightarrow S)$$

76 v 75

$$\vdash P \vee \neg P$$

Si se niega al menos uno de los consecuentes ($\rightarrow$ o V's), entonces debe negarse al menos uno de los antecedentes.